

Projektkonfiguration

Vanessa Künzel & Jasmin Hundt

Projektvision

Lernzeit-Manager ist eine Webanwendung zur Unterstützung von Studierenden bei der langfristigen Lernplanung, der Erfassung von Lernzeiten und der Verfolgung persönlicher Lernziele.

Die Anwendung soll Nutzer dabei unterstützen:

- Lernziele für sechs Monate zu definieren
- Lernzeiten langfristig und kurzfristig zu planen
- tatsächliche Lernzeiten zu erfassen
- Fortschritte visuell auszuwerten
- Erinnerungen bei ausstehenden Lernaktivitäten zu erhalten

Ziel ist die Verbesserung von Selbstorganisation, Motivation und Transparenz im Lernprozess.

Vorstellung des Teams

Name: Jasmin Hundt

Kurzprofil: Studentin im Studiengang Informatik

Rolle: Projektleitung, Backend Entwicklung

Skills: Java, JavaScript, SQL, Webentwicklung, Git, ABAP



Name: Vanessa Künzel

Kurzprofil: Studentin im Studiengang Informatik

Rolle: Frontend Entwicklung

Skills: Python, Java, SQL, GO



Gemeinsame Rollen: Architektur, Testing, Präsentation, Code Reviews, Dokumentation

Anforderungen

Funktionale Anforderungen

Die Anwendung ermöglicht eine Benutzerverwaltung mit Registrierung, Login und Logout. Nach der Anmeldung können Nutzer individuelle Lernziele anlegen, bearbeiten, löschen und deren Bearbeitungsstatus verwalten.

Zur Unterstützung des Lernprozesses bietet die Anwendung eine Lernzeitplanung auf verschiedenen Ebenen. Neben einer Grobplanung für einen Zeitraum von bis zu sechs Monaten können Monatspläne erstellt und einzelne Lernblöcke verwaltet werden.

Die tatsächliche Lernzeit wird über eine integrierte Zeiterfassung dokumentiert. Hierfür stehen Funktionen zum Starten, Pausieren und Stoppen eines Timers zur Verfügung. Die erfassten Lernzeiten werden dauerhaft gespeichert.

Darüber hinaus unterstützt die Anwendung die Fortschrittsverfolgung. Nutzer können erreichte Lernziele dokumentieren, ihre bisher investierten Lernstunden einsehen und den aktuellen Fortschritt visualisieren.

Um die Regelmäßigkeit des Lernens zu fördern, versendet das System Erinnerungen für geplante Lernzeiten sowie Benachrichtigungen bei längerer Inaktivität.

Ein Dashboard stellt die wichtigsten Informationen zentral bereit. Dazu gehören Visualisierungen der Lernzeiten, Übersichten zur Zielerreichung sowie verschiedene statistische Auswertungen.

Qualitätsanforderungen

Die Anwendung soll als responsive Webanwendung umgesetzt werden und sowohl auf Desktop- als auch auf mobilen Endgeräten nutzbar sein. Die Bedienung muss intuitiv gestaltet werden, sodass neue Benutzer ohne umfangreiche Einarbeitung mit der Anwendung arbeiten können. Antwortzeiten sollen im Regelfall unter zwei Sekunden liegen. Die Authentifizierung muss sicher erfolgen und der Quellcode soll wartbar, nachvollziehbar dokumentiert und erweiterbar sein.

Randbedingungen

Die Entwicklung erfolgt im Rahmen eines Studienprojekts durch ein Team von zwei Personen. Für die Bereitstellung und Ausführung der Anwendung wird Docker Compose eingesetzt. Die Versionsverwaltung und Zusammenarbeit erfolgen über GitHub. Das Ergebnis stellt eine prototypische Umsetzung der definierten Anforderungen dar.

Lösungsansatz

Die Anwendung wird als moderne, containerisierte Webanwendung umgesetzt, die aus einem Frontend, einem Backend sowie einer eingebetteten relationalen Datenbank besteht. Die einzelnen Systemkomponenten sind klar voneinander getrennt und kommunizieren über eine REST-Schnittstelle miteinander. Die gesamte Systemarchitektur ist auf einfache lokale Entwicklung, Wartbarkeit und Erweiterbarkeit ausgelegt.

Das Frontend wird mit React in Kombination mit Vite und TypeScript entwickelt. Für das Styling und die UI-Komponenten kommt Bootstrap zum Einsatz, um eine responsive und konsistente Benutzeroberfläche bereitzustellen. Die Anwendung fokussiert sich auf eine intuitive Bedienung zur Planung, Erfassung und Auswertung von Lernzeiten und Lernzielen.

Das Backend wird mit Node.js und dem NestJS-Framework umgesetzt. NestJS bietet dabei eine strukturierte Architektur für skalierbare Serveranwendungen und unterstützt die saubere Trennung von Modulen, Controllern und Services. Die Geschäftslogik, insbesondere zur Verwaltung von Lernzielen, Zeittracking und Fortschrittsberechnung, wird im Backend abgebildet und über eine REST API dem Frontend bereitgestellt. Zusätzlich wird Prisma als ORM eingesetzt, um einen effizienten und typsicheren Zugriff auf die Datenbank zu ermöglichen.

Als Datenbank wird SQLite verwendet, um eine leichtgewichtige und lokal lauffähige Persistenzlösung bereitzustellen. Diese speichert sämtliche relevanten Informationen wie Nutzerprofile, Lernziele, Zeitbuchungen und Fortschrittsdaten in strukturierter Form.

Die gesamte Anwendung wird mithilfe von Docker Compose orchestriert. Dadurch werden Frontend, Backend und Datenbank als voneinander isolierte Container betrieben, was eine konsistente Entwicklungsumgebung und eine einfache Inbetriebnahme des Gesamtsystems ermöglicht.

Für die Versionsverwaltung wird GitHub eingesetzt. Die Aufgabenplanung und Nachverfolgung erfolgt über GitHub Issues sowie ergänzend über Redmine, um eine strukturierte Projektorganisation und transparente Kommunikation innerhalb des Teams sicherzustellen.

Annahmen und Beschränkungen

Annahmen

- Benutzer verwalten ausschließlich ihre eigenen Daten.
- Erinnerungen können per E-Mail oder Browser-Benachrichtigung erfolgen.
- Ein Nutzer verwendet die Anwendung auf einem Gerät gleichzeitig.

Beschränkungen

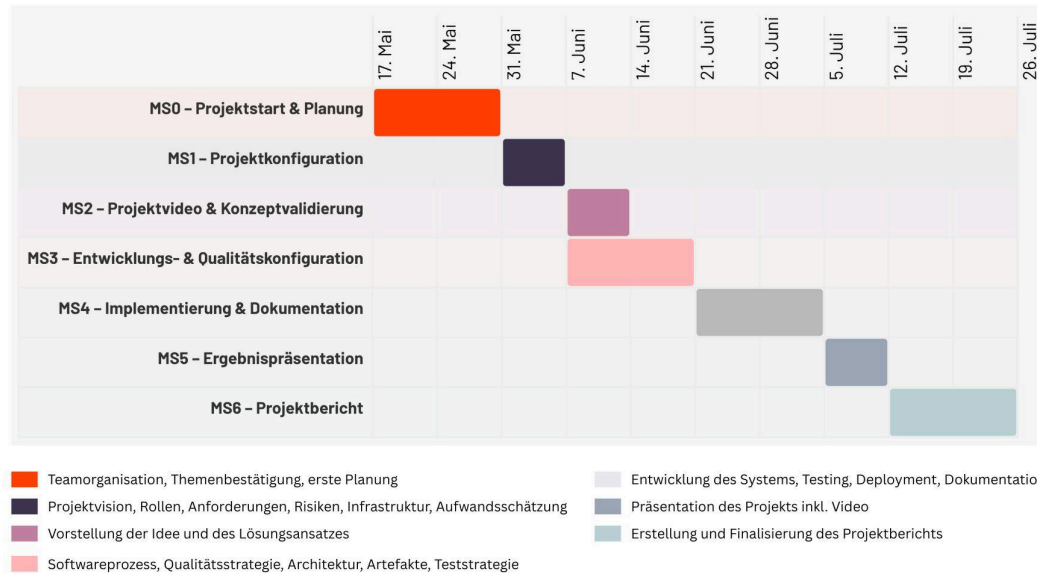
- Keine mobile App
- Keine Kalenderintegration (Google Calendar etc.)
- Keine Mehrbenutzerfunktionen
- Keine KI-Funktionalitäten

Meilensteinplan

Projekt Software Engineering: Lernzeit-Manager

Vanessa Künzel & Jasmin Hundt

Meilensteinplan



Liefergegenstände

Liefergegenstand	Meilenstein
Projektkonfiguration	MS1
Anforderungsdokument	MS2
Architekturkonzept	MS2
Datenmodell	MS2
Implementierter Prototyp	MS3/MS4
Testdokumentation	MS4
Benutzerhandbuch	MS5
Projektdokumentation	MS5

Projektstrukturplan

Das Projekt ist in mehrere Phasen und Funktionsbereiche untergliedert, die den gesamten Entwicklungsprozess von der Planung bis zur Dokumentation abbilden.

Im Bereich **Projektmanagement** werden die organisatorischen Grundlagen des Projekts festgelegt. Dazu gehören die Projektplanung, die Aufwandsschätzung sowie das Risikomanagement. Ergänzend finden regelmäßige Meetings zur Abstimmung und Koordination im Team statt.

Die **Anforderungsanalyse** bildet die Grundlage für die fachliche Ausarbeitung des Systems. Hier werden Use Cases und User Stories definiert sowie erste Mockups zur Visualisierung der Benutzeroberfläche erstellt.

Im Rahmen der **Architektur** wird das technische Gesamtsystem entworfen. Dies umfasst die Systemarchitektur, das Datenmodell sowie das API-Design zur Kommunikation zwischen Frontend und Backend.

Die Entwicklung des **Frontends** konzentriert sich auf die Umsetzung der Benutzeroberfläche. Dazu gehören die Authentifizierung mit Login und Registrierung, ein Dashboard zur zentralen Übersicht sowie Funktionen zur Darstellung von Statistiken und Visualisierungen. Zusätzlich werden Module für die Lernplanung, Lernzielverwaltung, Lernzeitverwaltung und Zeiterfassung inklusive Timer und Historie umgesetzt.

Das **Backend** stellt die fachliche Logik der Anwendung bereit. Hier werden Funktionen für die Benutzerverwaltung, Lernziele, Lernplanung und Lernzeiterfassung implementiert. Zusätzlich wird ein Erinnerungsservice entwickelt, der Nutzer aktiv an geplante Lernzeiten erinnert.

Die **Datenbank** dient der strukturierten Speicherung aller relevanten Daten. Hierzu werden das Datenmodell entworfen, Migrationen durchgeführt und Testdaten zur Entwicklung bereitgestellt.

Im Bereich **Testing** werden verschiedene Qualitätssicherungsmaßnahmen durchgeführt. Dazu zählen Unit Tests, Integrationstests sowie Abnahmetests zur Überprüfung der gesamten Anwendung.

Abschließend wird im Bereich **Dokumentation** sowohl die technische Dokumentation als auch die Projektdokumentation erstellt. Zusätzlich wird eine abschließende Präsentation zur Vorstellung der Ergebnisse vorbereitet.

Aufwandsschätzung

Bereich	Stunden
Projektmanagement	20

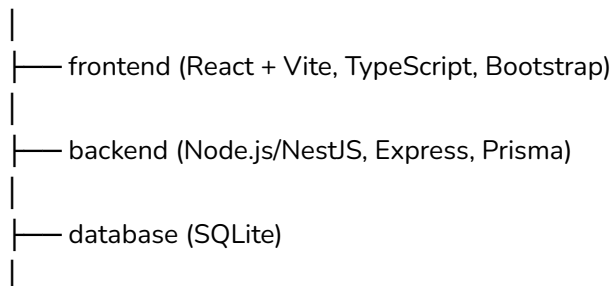
Analyse	20
Architektur	15
Frontend	60
Backend	60
Datenbank	15
Testing	20
Dokumentation	20
Gesamt	230

Eingesetzte Systeme

Zweck	System
Versionsverwaltung	GitHub
Quellcode	GitHub Repository
Aufgabenverwaltung	Redmine
Dokumentation	Markdown + PDF
UI-Design	Figma
Entwicklung	VS Code
Datenbank	PostgreSQL
Containerisierung	Docker Compose

Aufbau der technischen Infrastruktur

GitHub Repository



└─ docker-compose.yml

Personalmanagement

Verfügbare Kapazität: Teammitglied 1 & 2: ca. 20 Stunden/Woche

Urlaubs- und Abwesenheitsplanung: Abwesenheiten werden frühzeitig im Redmine-Kalender dokumentiert. Derzeit sind keine bekannt.

Kommunikationsmanagement

Wöchentliches Sprint-Meeting: Sonntag 13:00 Uhr (30-120 min)

Kommunikationswege

Zweck	Werkzeug
Kurze Abstimmungen	Teams Chat
Aufgabenverwaltung	Redmine/ ClickUp
Quellcode	GitHub
Meetings	Teams

Risikomanagement

Risiko	Wahrscheinlichkeit	Auswirkung	Maßnahme
Zeitmangel	Mittel	Hoch	Pufferzeiten einplanen
Technische Probleme	Mittel	Mittel	Frühe Prototypen erstellen
Ausfall eines Teammitglieds	Niedrig	Hoch	Wissen verteilen
Unterschätzter Aufwand	Hoch	Mittel	Priorisierung der Anforderungen